



TerraOrbit

Project Report & Technical Documentation

An End-to-End AWS Infrastructure-as-Code Project Built with Terraform

Submitted for the TrainWithShubham #TerraWeek Challenge

Challenge Duration: July 12 – July 17, 2026

Platform: AWS Free Tier | Tooling: Terraform, AWS CLI, GitHub Actions

Demo Video: <https://youtu.be/lM-zPxM4NDM>

Repository: Public GitHub Repository (see References)

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Project Objectives.....	3
3. Architecture Overview	3
3.1 Networking Layer	3
3.2 Compute & Storage Layer	3
3.3 State & Automation Layer	3
4. Tools & Technologies Used.....	4
5. Day-Wise Implementation Summary	5
6. Detailed Day-Wise Log.....	6
Day 1 — Introduction to Terraform	6
Day 2 — Terraform Configuration Language (HCL)	6
Day 3 — Managing Resources.....	6
Day 4 — Terraform State Management.....	6
Day 5 — Terraform Modules	6
Day 6 — Terraform Providers	7
Day 7 — Advanced Terraform Topics	7
7. Issues Encountered & Resolutions	8
8. Cost & Free-Tier Compliance	9
9. Results & Outcomes	9
10. Conclusion & Key Learnings.....	9
11. Acknowledgements	9
12. References & Links.....	9

1. Executive Summary

TerraOrbit is an end-to-end, production-style AWS infrastructure project built entirely with Terraform over the course of the TrainWithShubham #TerraWeek Challenge (July 12–17, 2026). The project was designed to map directly onto the seven daily topics of the challenge — from Terraform fundamentals through to advanced workspace and CI/CD practices — while producing a single, coherent, working piece of infrastructure rather than seven disconnected exercises.

Over seven days, the project grew from a single provider configuration into a modular codebase provisioning a custom VPC, a live web server, S3 storage, remote state management with locking, multi-region provider configuration, environment-isolated workspaces, and an automated GitHub Actions deployment pipeline — all while remaining within AWS Free Tier limits.

This document records the objectives, architecture, day-wise implementation, issues encountered during development, and the outcomes of the project, and serves as the formal submission report accompanying the public GitHub repository.

2. Project Objectives

- Build one real, deployable AWS project — not isolated demos — covering all seven #TerraWeek topics.
- Stay entirely within AWS Free Tier eligible resources to avoid any billing risk.
- Practice Infrastructure-as-Code fundamentals: providers, resources, variables, state, and modules.
- Introduce production practices: remote state locking, environment isolation, and CI/CD automation.
- Document the build process day-by-day with reproducible commands and screenshots for public submission.

3. Architecture Overview

TerraOrbit provisions the following AWS resources, organized into three logical layers:

3.1 Networking Layer

- A custom VPC (10.0.0.0/16)
- 2 public subnets and 2 private subnets across 2 Availability Zones
- An Internet Gateway and public route table with associations
- A Security Group permitting SSH (restricted to the developer's IP) and HTTP (public)

3.2 Compute & Storage Layer

- An EC2 instance (t2.micro, Amazon Linux 2023) running Nginx, provisioned via a user_data bootstrap script
- An S3 bucket for static assets, versioned and uniquely named via the random provider

3.3 State & Automation Layer

- A dedicated S3 bucket storing remote Terraform state, versioned for recoverability
- A DynamoDB table providing state locking to prevent concurrent-apply corruption
- Three Terraform workspaces (dev, stage, prod) for environment isolation
- A GitHub Actions pipeline running fmt, validate, plan, and apply on every push

All resources used (VPC, t2.micro EC2, S3, DynamoDB on-demand) fall within AWS Free Tier limits for low-volume, learning-scale usage.

4. Tools & Technologies Used

- Terraform ($\geq 1.7.0$) — Infrastructure as Code
- AWS Provider (~ 5.0) and Random Provider (~ 3.6)
- AWS services: VPC, EC2, S3, DynamoDB, IAM
- AWS CLI v2 — verification and manual inspection
- Git & GitHub — version control and public repository hosting
- GitHub Actions — CI/CD pipeline automation
- TFLint — static analysis and best-practice linting

5. Day-Wise Implementation Summary

The table below maps each day of the challenge to the Terraform topic covered and the work completed. Full command-by-command detail for each day is maintained in `Project-Plan.md` within the repository.

Day	Date	Topic	Summary
Day 1	Sunday	Introduction to Terraform	Installed Terraform & AWS CLI, configured credentials, wrote the first <code>provider.tf</code> , ran <code>init/plan/apply</code> .
Day 2	Monday	HCL — Variables & Expressions	Built <code>variables.tf</code> and <code>locals.tf</code> ; practiced data types, string interpolation, and conditionals via terraform console.
Day 3	Tuesday	Managing Resources	Created VPC, subnets, IGW, route tables, security group, and an EC2 instance running a live Nginx web server.
Day 4	Wednesday	State Management	Bootstrapped an S3 backend + DynamoDB lock table; migrated to remote state; tested state locking.
Day 5	Thursday	Terraform Modules	Refactored the flat configuration into network, compute, and s3-website modules with wired outputs/inputs.
Day 6	Friday (Pt.1)	Terraform Providers	Pinned provider versions, added a second aliased AWS provider (multi-region), used the random provider.
Day 7	Friday (Pt.2)	Advanced Topics	Created dev/stage/prod workspaces and built a GitHub Actions CI/CD pipeline (<code>fmt</code> , <code>validate</code> , <code>plan</code> , <code>apply</code>).

6. Detailed Day-Wise Log

Day 1 — Introduction to Terraform

Objective: understand Infrastructure as Code fundamentals and complete the first successful Terraform run.

- Installed Terraform and the AWS CLI; configured IAM credentials via `aws configure`.
- Initialized the project repository and `.gitignore`.
- Wrote `provider.tf` targeting the `ap-south-1` (Mumbai) region.
- Added a data `"aws_caller_identity"` block and output to confirm connectivity.
- Ran `terraform init`, `plan`, and `apply` successfully for the first time.

Files added: provider.tf, main.tf, outputs.tf

Day 2 — Terraform Configuration Language (HCL)

Objective: build fluency with HCL syntax, data types, and expressions.

- Defined `variables.tf` covering strings, lists, and maps (`project_name`, `vpc_cidr`, subnet CIDR lists, tag maps).
- Defined `locals.tf` with a `merge()`-based common tagging strategy, string interpolation for naming, and a conditional (ternary) expression.
- Practiced `for-expressions` and local values interactively using `terraform console`.
- Adopted `terraform fmt` and `terraform validate` as a standing daily practice.

Files added: variables.tf, locals.tf

Day 3 — Managing Resources

Objective: define and manage real, interdependent AWS resources.

- Built the VPC, Internet Gateway, public/private subnets, and route table with associations.
- Configured a Security Group with a `create_before_destroy` lifecycle rule.
- Provisioned an EC2 instance using the latest Amazon Linux 2023 AMI (resolved dynamically via a data source), bootstrapped with Nginx via `user_data`.
- Used `depends_on` to sequence the instance launch after route table association.
- Verified the deployment via `curl` and browser access to the instance's public IP.

Files added: network.tf, security.tf, compute.tf

Day 4 — Terraform State Management

Objective: move from local state to a shared, locked remote backend.

- Bootstrapped a standalone configuration to create a versioned S3 bucket and a DynamoDB locking table, isolated from the main project's lifecycle.
- Configured `backend.tf` to point the main project at the new S3/DynamoDB backend.
- Migrated existing state using `terraform init -migrate-state`.
- Practiced `terraform state list`, `state show`, and `state mv`.
- Verified locking behaviour by running concurrent `terraform plan` commands in two terminals.

Files added: bootstrap/main.tf, backend.tf

Day 5 — Terraform Modules

Objective: refactor working infrastructure into reusable, composable modules.

- Split Day 3's flat resource files into three modules: network, compute, and s3-website.
- Rebuilt the root main.tf to call each module, wiring module outputs into subsequent module inputs (e.g., subnet IDs flowing from network into compute).
- Documented the path to module versioning via Git-tag-pinned module sources for future stability.

Files added: modules/network/, modules/compute/*, modules/s3-website/**

Day 6 — Terraform Providers

Objective: harden provider configuration, versioning, and authentication.

- Added versions.tf pinning hashicorp/aws (~> 5.0) and hashicorp/random (~> 3.6).
- Added a second, aliased AWS provider (alias = "dr") targeting us-east-1 for future multi-region use.
- Used the random provider to generate globally unique S3 bucket suffixes.
- Verified provider wiring using terraform providers.

Files added: versions.tf; updated provider.tf

Day 7 — Advanced Terraform Topics

Objective: introduce environment isolation and deployment automation.

- Created dev, stage, and prod workspaces and parameterized naming via terraform.workspace.
- Built a GitHub Actions pipeline (.github/workflows/terraform.yml) running fmt, validate, and plan on pull requests, and apply automatically on merges to main.
- Stored AWS credentials as encrypted GitHub Actions repository secrets.
- Finalized formatting, validation, and project documentation.

Files added: .github/workflows/terraform.yml, README.md, Project-Plan.md

7. Issues Encountered & Resolutions

Building a CI/CD-integrated project surfaced several real-world issues beyond the core Terraform concepts — most of them only visible once the code ran somewhere other than the original development machine. These are documented below for transparency and as a reference for future debugging.

Issue	Root Cause	Resolution
tf lint unused declaration warnings	local.is_prod, provider alias aws.dr, and var.region were declared but never referenced.	Wired is_prod into instance sizing, passed aws.dr through the module providers block, and used var.region inside provider.tf.
Invalid AWS Region: var.region	Region value was wrapped in quotes ("var.region") instead of referencing the variable.	Changed to region = var.region (no quotes) so the variable resolves correctly.
Provider configuration not present	A resource created via the aws.dr aliased provider existed in state after the alias was removed from provider.tf.	Re-added the alias, passed it through the module's providers block, and added configuration_aliases in the module's required_providers.
terraform fmt -check failing in CI	Local files were not run through terraform fmt before committing.	Ran terraform fmt -recursive locally and committed the corrected formatting.
State/provider attribute decode warnings in CI	.terraform.lock.hcl was git-ignored, so CI resolved a different provider version than local.	Removed .terraform.lock.hcl from .gitignore and committed it so CI and local always use identical provider versions.
CI applied into the wrong workspace	GitHub Actions defaulted to the "default" workspace instead of "dev".	Added a workspace select step in the pipeline: terraform workspace select dev terraform workspace new dev.
Invalid index on public_subnet_ids[0]	The network module's public subnet count evaluated to zero because subnet CIDRs weren't passed through from the root module.	Explicitly passed public_subnet_cidrs and private_subnet_cidrs into the network module call.

8. Cost & Free-Tier Compliance

Every resource in TerraOrbit was deliberately chosen to remain within AWS Free Tier limits:

- EC2: a single t2.micro instance (covered under the 750 free hours/month tier).
- S3: total storage well under the 5 GB free tier allowance.
- DynamoDB: on-demand billing mode with negligible read/write volume for state locking.
- VPC, subnets, route tables, security groups, and IAM: no cost at any usage level.

A full destroy sequence (terraform destroy across all three workspaces, followed by destroying the bootstrap backend) is documented in the README to guarantee no residual charges after evaluation.

9. Results & Outcomes

- A single, working Terraform codebase provisions a complete network, compute, and storage stack on AWS.
- Remote state with locking eliminates the single-user, single-machine limitation of local state.
- Modular structure makes the network and compute layers reusable across environments.
- A working CI/CD pipeline automatically validates and can deploy changes on every push to main.
- The live web server, remote state backend, and CI pipeline were all verified working via screenshots included in the project README.

10. Conclusion & Key Learnings

The #TerraWeek Challenge format — one topic a day, compounding into a single project — proved far more effective than isolated tutorials. State management in particular stood out: understanding why local state doesn't scale, and how remote locking prevents corruption, only became clear by deliberately testing concurrent applies. Refactoring into modules mid-project also reinforced how Terraform benefits from being treated as software, with clear interfaces between components, rather than a flat script.

Beyond Terraform syntax, integrating GitHub Actions surfaced an entirely different category of problem — version drift between local and CI environments, workspace assumptions, and lint hygiene — all of which are only visible once a project is shared and automated rather than run solely on one developer's machine.

11. Acknowledgements

This project would not have been possible without the guidance of:

- Shubham Bhaiya (TrainWithShubham) — for teaching Terraform from absolute scratch and designing the #TerraWeek Challenge structure that made a project of this scope achievable within a single week.
- Gangadhar Bhaiya — for hands-on help with the AWS fundamentals (VPC, EC2, IAM) underpinning this project's cloud infrastructure.

12. References & Links

- Demo Video: <https://youtu.be/lM-zPxM4NDM>
- Project Plan: Project-Plan.md (repository root)
- README with Architecture & Screenshots: README.md (repository root)
- Challenge Host: TrainWithShubham — #TerraWeek Challenge